

Lab5

组员：杨桑多杰，张磊，徐南海

练习0：填写已有实验

解答：

更新一下Lab4的初始化代码：

位于 lab5/kern/process/proc.c 的 alloc_proc() 函数：

```
static struct proc_struct *
alloc_proc(void)
{
    struct proc_struct *proc = kmalloc(sizeof(struct proc_struct));
    if (proc != NULL)
    {
        proc->state = PROC_UNINIT;
        proc->pid = -1;
        proc->runs = 0;
        proc->kstack = 0;
        proc->need_resched = 0;
        proc->parent = NULL;
        proc->mm = NULL;
        memset(&(proc->context), 0, sizeof(struct context));
        proc->tf = NULL;
        proc->pgdir = boot_pgdir_pa;
        proc->flags = 0;
        memset(proc->name, 0, PROC_NAME_LEN);
        list_init(&(proc->list_link));
        list_init(&(proc->hash_link));
        // LAB5 YOUR CODE : (update LAB4 steps)
        /*
         * below fields(add in LAB5) in proc_struct need to be initialized
         *      uint32_t wait_state;                // waiting state
         *      struct proc_struct *cptr, *yptr, *optr;    // relations between processes
         */
        proc->wait_state = 0;
        proc->cptr = NULL;
        proc->optr = NULL;
        proc->yptr = NULL;
    }
    return proc;
}
```

在已有的函数上，添加 wait_state、*cptr, *yptr, *optr 的初始化操作

位于 lab5/kern/process/proc.c 的 do_fork() 函数：

```

int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
{
    int ret = -E_NO_FREE_PROC;
    struct proc_struct *proc;
    if (nr_process >= MAX_PROCESS)
    {
        goto fork_out;
    }
    ret = -E_NO_MEM;
    proc = alloc_proc();
    if (proc == NULL)
    {
        goto fork_out;
    }
    proc->parent = current;
    assert(current->wait_state == 0);
    if (setup_kstack(proc) != 0)
    {
        goto bad_fork_cleanup_proc;
    }
    if (copy_mm(clone_flags, proc) != 0)
    {
        goto bad_fork_cleanup_kstack;
    }
    copy_thread(proc, stack, tf);
    proc->pid = get_pid();
    hash_proc(proc);
    set_links(proc);
    wakeup_proc(proc);
    ret = proc->pid;
fork_out:
    return ret;

bad_fork_cleanup_kstack:
    put_kstack(proc);
bad_fork_cleanup_proc:
    kfree(proc);
    goto fork_out;
}

```

在之前的基础上添加 `proc->parent = current`，将当前的进程设置为子进程的父进程。`assert(current->wait_state == 0)` 确保了当前进程的等待状态为0。`set_links(proc)` 设置了进程间的关系。

练习1: 加载应用程序并执行（需要编码）

`do_execve`函数调用 `load_icode`（位于 `kern/process/proc.c`中）来加载并解析一个处于内存中的ELF执行文件格式的应用程序。你需要补充 `load_icode` 的第6步，建立相应的用户内存空间来放置应用程序的代码段、数据段等，且要设置好 `proc_struct` 结构中的成员变量 `trapframe` 中的内容，确保在执行此进程后，能够从应用程序设定的起始执行地址开始执行。需设置正确的 `trapframe` 内容。

请在实验报告中简要说明你的设计实现过程。

- 请简要描述这个用户态进程被ucore选择占用CPU执行（RUNNING态）到具体执行应用程序第一条指令的整个经过。

解答：

```
// 设置用户栈顶指针
tf->gpr.sp = USTACKTOP;

// 设置程序入口点（ELF 文件的入口地址）
tf->epc = elf->e_entry;

// 设置状态寄存器：
// SSTATUS_SPP (Supervisor Previous Privilege): 设置为0，表示之前的特权级是User Mode，sret后将回到User Mode
// SSTATUS_SPIE (Supervisor Previous Interrupt Enable): 设置为1，表示中断使能，允许用户态响应中断
tf->status = sstatus & ~SSTATUS_SPP;
tf->status |= SSTATUS_SPIE;
```

我们将 sp 寄存器设置为栈顶 USTACKTOP，然后将 epc 寄存器设置为文件的入口地址，将 sstatus 的 SPP 位清零，表示异常来自用户态；同时将 sstatus 的 SPIE 位设置为1，表示允许用户态响应中断。

执行过程

- 1、首先在 proc_init() 初始化的第一个内核进程 idleproc 中，我们执行 init_main() 函数，通过 int pid = kernel_thread(user_main, NULL, 0) 来调用 do_fork() 来创建新的内核进程，并把其状态唤醒为 PROC_RUNNABLE，然后执行函数 user_main()，这个内核进程里我们将要开始执行用户进程
- 2、user_main() 函数执行 kern_execve() 函数，加载了存储在这个位置的程序 exit 并在 user_main 这个进程里开始执行。这时 user_main 就从内核进程变成了用户进程
- 3、kern_execve() 中我们用 ebreak 产生断点中断进行处理，通过设置 a7 寄存器的值为10说明这不是一个普通的断点中断，而是要转发到 syscall()，具体实现为发生断点异常后，系统转到 __alltraps，转到 trap，再到 trap_dispatch，然后到 exception_handler，再到 CAUSE_BREAKPOINT 处，最后调用 syscall 函数
- 4、经过中断处理流程中一系列的函数跳转，最终进入到 _exec 的系统处理函数 do_execve 中，在 do_execve 中调用 load_icode 加载应用程序
- 5、加载完毕后返回，直至 __alltraps 的末尾，接着执行 __trapret 后的内容，到 sret，表示退出S态，回到U态执行，这时开始执行用户的应用程序

练习2: 父进程复制自己的内存空间给子进程（需要编码）

创建子进程的函数 `do_fork` 在执行中将拷贝当前进程（即父进程）的用户内存地址空间中的合法内容到新进程中（子进程），完成内存资源的复制。具体是通过 `copy_range` 函数（位于 `kern/mm/pmm.c` 中）实现的，请补充 `copy_range` 的实现，确保能够正确执行。

请在实验报告中简要说明你的设计实现过程。

- 如何设计实现 Copy on Write 机制？给出概要设计，鼓励给出详细设计。

解答：

`copy_range()` 函数的具体实现如下：

```
int copy_range(pde_t *to, pde_t *from, uintptr_t start, uintptr_t end,
               bool share)
{
    assert(start % PGSIZE == 0 && end % PGSIZE == 0);
    assert(USER_ACCESS(start, end));
    do
    {
        pte_t *ptep = get_pte(from, start, 0), *nptep;
        if (ptep == NULL)
        {
            start = ROUNDDOWN(start + PTSIZE, PTSIZE);
            continue;
        }
        if (*ptep & PTE_V)
        {
            if ((nptep = get_pte(to, start, 1)) == NULL)
            {
                return -E_NO_MEM;
            }
            uint32_t perm = (*ptep & PTE_USER);
            struct Page *page = pte2page(*ptep);
            struct Page *npage = alloc_page();
            assert(page != NULL);
            assert(npage != NULL);
            int ret = 0;

            // (1) 获取源页面（父进程）的内核虚拟地址
            uintptr_t *src_kvaddr = page2kva(page);

            // (2) 获取目标页面（子进程）的内核虚拟地址
            uintptr_t *dst_kvaddr = page2kva(npage);

            // (3) 将源页面的内容复制到目标页面，复制大小为一个页面（PGSIZE）
            memcpy(dst_kvaddr, src_kvaddr, PGSIZE);

            // (4) 建立目标页面的物理地址与线性地址 start 的映射关系
            ret = page_insert(to, npage, start, perm);
        }
    } while (start < end);
    return ret;
}
```

```

        assert(ret == 0);
    }
    start += PGSIZE;
} while (start != 0 && start < end);
return 0;
}

```

具体的实现：

1. 首先获取 `src_kvaddr` 源地址和 `dst_kvaddr` 目的地址的内核虚拟地址；
2. 拷贝内存，将 `src_kvaddr` 的内存复制到 `dst_kvaddr` 中；
3. 最后将拷贝完的页插入到页表中即可。

练习3: 阅读分析源代码，理解进程执行 `fork/exec/wait/exit` 的实现，以及系统调用的实现（不需要编码）

请在实验报告中简要说明你对 `fork/exec/wait/exit` 函数的分析。并回答如下问题：

- 请分析 `fork/exec/wait/exit` 的执行流程。重点关注哪些操作是在用户态完成，哪些是在内核态完成？内核态与用户态程序是如何交错执行的？内核态执行结果是如何返回给用户程序的？
- 请给出 `ucore` 中一个用户态进程的执行状态生命周期图（包执行状态，执行状态之间的变换关系，以及产生变换的事件或函数调用）。（字符方式画即可）

解答：

`sys_fork()` 函数

```

static int
sys_fork(uint64_t arg[]) {
    struct trapframe *tf = current->tf;
    uintptr_t stack = tf->gpr.sp;
    return do_fork(0, stack, tf);
}

```

`sys_fork` 函数调用 `do_fork` 函数。 `do_fork` 函数如下所示：

```

int
do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf) {
    int ret = -E_NO_FREE_PROC;
    struct proc_struct *proc;
    if (nr_process >= MAX_PROCESS) {
        goto fork_out;
    }
    ret = -E_NO_MEM;
    if ((proc = alloc_proc()) == NULL)
    {
        goto fork_out;
    }
}

```

```

}
proc->parent = current; // 添加
assert(current->wait_state == 0);
if(setup_kstack(proc) != 0)
{
    goto bad_fork_cleanup_proc;
}
;
if(copy_mm(clone_flags, proc) != 0)
{
    goto bad_fork_cleanup_kstack;
}
copy_thread(proc, stack, tf);
bool intr_flag;
local_intr_save(intr_flag);
{
    int pid = get_pid();
    proc->pid = pid;
    hash_proc(proc);
    set_links(proc);
}
local_intr_restore(intr_flag);
wakeup_proc(proc);
ret = proc->pid;

fork_out:
    return ret;

bad_fork_cleanup_kstack:
    put_kstack(proc);
bad_fork_cleanup_proc:
    kfree(proc);
    goto fork_out;
}

```

首先，为该进程的父进程赋值为当前的进程，`setup_kstack` 完成了内核栈空间的分配；`copy_mm` 完成了分配新的虚拟内存或与其他线程共享虚拟内存；`copy_thread` 获取了原线程的上下文与中断帧，并且设置了当前线程的上下文与中断帧；然后，为新进程获取新的 `pid` 进程号，并且赋值给该进程。然后将新线程插入哈希表和链表中，唤醒该新建的进程，返回该进程的 `pid` 号。

`sys_exec()` 函数

```

static int
sys_exec(uint64_t arg[]) {
    const char *name = (const char *)arg[0];
    size_t len = (size_t)arg[1];
    unsigned char *binary = (unsigned char *)arg[2];
    size_t size = (size_t)arg[3];
    return do_execve(name, len, binary, size);
}

```

sys_exec 函数调用了 do_execve 函数。

```

int
do_execve(const char *name, size_t len, unsigned char *binary, size_t size) {
    struct mm_struct *mm = current->mm;
    if (!user_mem_check(mm, (uintptr_t)name, len, 0)) {
        return -E_INVALID;
    }
    if (len > PROC_NAME_LEN) {
        len = PROC_NAME_LEN;
    }

    char local_name[PROC_NAME_LEN + 1];
    memset(local_name, 0, sizeof(local_name));
    memcpy(local_name, name, len);

    if (mm != NULL) {
        cputs("mm != NULL");
        lcr3(boot_cr3);
        if (mm_count_dec(mm) == 0) {
            exit_mmap(mm);
            put_pgdir(mm);
            mm_destroy(mm);
        }
        current->mm = NULL;
    }
    int ret;
    if ((ret = load_icode(binary, size)) != 0) {
        goto execve_exit;
    }
    set_proc_name(current, local_name);
    return 0;

execve_exit:
    do_exit(ret);
    panic("already exit: %e.\n", ret);
}

```

该函数用于创建用户空间，加载用户程序。完成了当前线程的虚拟内存空间的回收，以及为当前线程分配新的虚拟内存空间，并加载了应用程序。

sys_wait() 函数

```
static int
sys_wait(uint64_t arg[]) {
    int pid = (int)arg[0];
    int *store = (int *)arg[1];
    return do_wait(pid, store);
}
```

sys_wait 函数调用了 do_wait 函数。

```
int
do_wait(int pid, int *code_store) {
    struct mm_struct *mm = current->mm;
    if (code_store != NULL) {
        if (!user_mem_check(mm, (uintptr_t)code_store, sizeof(int), 1)) {
            return -EINVAL;
        }
    }

    struct proc_struct *proc;
    bool intr_flag, haskid;
repeat:
    haskid = 0;
    if (pid != 0) {
        proc = find_proc(pid);
        if (proc != NULL && proc->parent == current) {
            haskid = 1;
            if (proc->state == PROC_ZOMBIE) {
                goto found;
            }
        }
    }
    else {
        proc = current->cptr;
        for (; proc != NULL; proc = proc->optr) {
            haskid = 1;
            if (proc->state == PROC_ZOMBIE) {
                goto found;
            }
        }
    }
    if (haskid) {
        current->state = PROC_SLEEPING;
        current->wait_state = WT_CHILD;
        schedule();
        if (current->flags & PF_EXITING) {
            do_exit(-E_KILLED);
        }
    }
}
```



```

    }
    goto repeat;
}
return -E_BAD_PROC;

found:
if (proc == idleproc || proc == initproc) {
    panic("wait idleproc or initproc.\n");
}
if (code_store != NULL) {
    *code_store = proc->exit_code;
}
local_intr_save(intr_flag);
{
    unhash_proc(proc);
    remove_links(proc);
}
local_intr_restore(intr_flag);
put_kstack(proc);
kfree(proc);
return 0;
}

```

首先，查找状态为 `PROC_ZOMBIE` 的子线程；如果找到了，就将线程从哈希表和链表中删除，最后释放线程的资源。

如果查询到拥有子线程的线程，则设置线程状态并切换线程；如果线程已退出，则调用 `do_exit` 函数。

`sys_exit()` 函数

```

static int
sys_exit(uint64_t arg[]) {
    int error_code = (int)arg[0];
    return do_exit(error_code);
}

```

`sys_exit` 函数调用了 `do_exit` 函数。

```

int
do_exit(int error_code) {
    if (current == idleproc) {
        panic("idleproc exit.\n");
    }
    if (current == initproc) {
        panic("initproc exit.\n");
    }
    struct mm_struct *mm = current->mm;
    if (mm != NULL) {
        lcr3(boot_cr3);
        if (mm_count_dec(mm) == 0) {

```

```

        exit_mmap(mm);
        put_pgdire(mm);
        mm_destroy(mm);
    }
    current->mm = NULL;
}
current->state = PROC_ZOMBIE;
current->exit_code = error_code;
bool intr_flag;
struct proc_struct *proc;
local_intr_save(intr_flag);
{
    proc = current->parent;
    if (proc->wait_state == WT_CHILD) {
        wakeup_proc(proc);
    }
    while (current->cptr != NULL) {
        proc = current->cptr;
        current->cptr = proc->optr;

        proc->yptr = NULL;
        if ((proc->optr = initproc->cptr) != NULL) {
            initproc->cptr->yptr = proc;
        }
        proc->parent = initproc;
        initproc->cptr = proc;
        if (proc->state == PROC_ZOMBIE) {
            if (initproc->wait_state == WT_CHILD) {
                wakeup_proc(initproc);
            }
        }
    }
}
local_intr_restore(intr_flag);
schedule();
panic("do_exit will not return!! %d.\n", current->pid);
}

```

具体代码逻辑：

- 如果当前线程的虚拟内存没有用于其他线程，则销毁该虚拟内存
- 如果用于其他的线程了，就将当前线程状态设为 `PROC_ZOMBIE`，唤醒该线程的父线程
- 完成 `exit` 后，调用 `schedule` 切换到其他线程

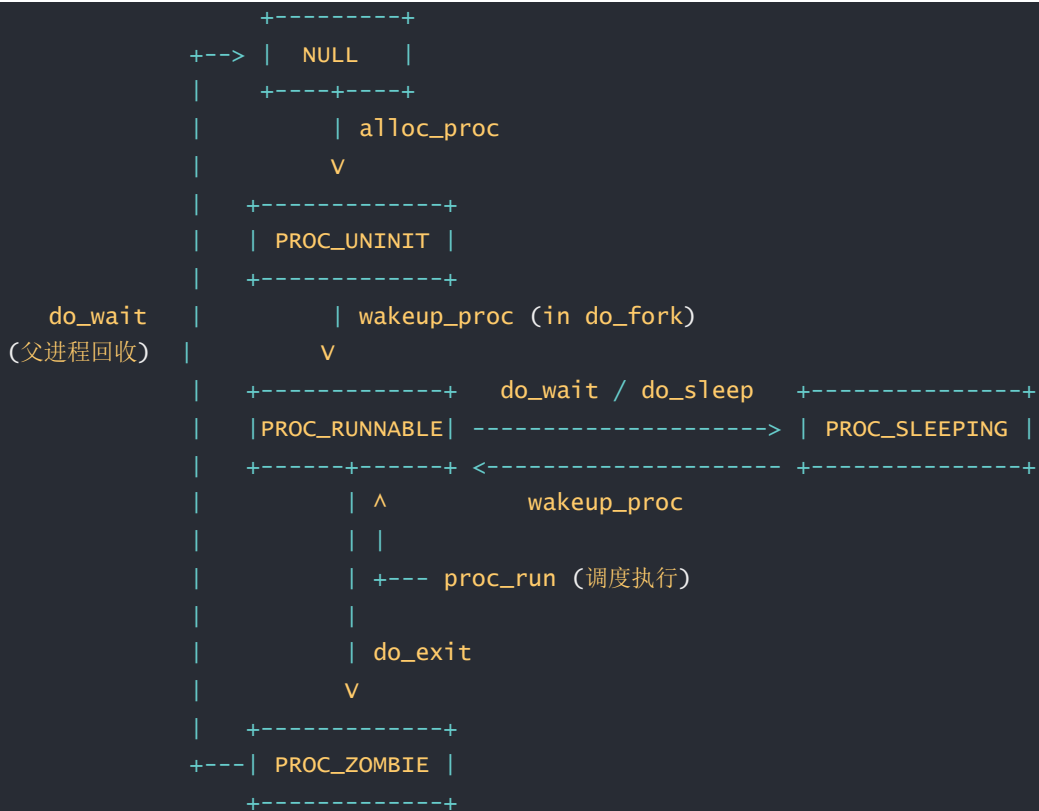
执行状态生命周期图

 ucore 用户态进程状态生命周期图

```

```text

```



## 状态转换说明

### 1. NULL -> PROC\_UNINIT:

- 事件: `alloc_proc`
- 说明: 进程控制块 (`proc_struct`) 被分配, 状态初始化为 `PROC_UNINIT`。

### 2. PROC\_UNINIT -> PROC\_RUNNABLE:

- 事件: `wakeup_proc` (通常在 `do_fork` 中调用)
- 说明: 进程初始化完成 (分配栈、页表等), 被添加到运行队列, 状态变为可运行。

### 3. PROC\_RUNNABLE <-> PROC\_RUNNABLE:

- 事件: `proc_run` / `schedule`
- 说明: 进程被调度器选中, 正在 CPU 上执行。在 `ucore` 中, 正在运行的进程状态依然标记为 `PROC_RUNNABLE`。

### 4. PROC\_RUNNABLE -> PROC\_SLEEPING:

- 事件: `do_wait`, `do_sleep`
- 说明: 进程等待子进程退出或主动休眠, 释放 CPU, 进入睡眠状态。

### 5. PROC\_SLEEPING -> PROC\_RUNNABLE:

- 事件: `wakeup_proc`
- 说明: 等待的条件满足 (如子进程退出、定时器到期), 进程被唤醒, 重新进入运行队列。

### 6. PROC\_RUNNABLE -> PROC\_ZOMBIE:

- 事件: `do_exit`
- 说明: 进程执行结束或异常终止, 释放大部分资源, 变成僵尸进程等待父进程回收。

## 7. PROC\_ZOMBIE -> NULL:

- **事件:** `do_wait` (由父进程调用)
- **说明:** 父进程回收僵尸子进程的剩余资源 (如内核栈、`proc_struct`) , 进程彻底消失。

最终得分: Total Score: 130/130, 完成实验

```
exit: (1.5s)
 -check result: OK
 -check output: OK
spin: (4.4s)
 -check result: OK
 -check output: OK
forktest: (1.3s)
 -check result: OK
 -check output: OK
Total Score: 130/130
❖ ed@DESKTOP-869CMET:~/Operating-System/lab5$ █
```

## 拓展练习Challenge

### 1. 实现 Copy on Write (COW) 机制

给出实现源码,测试用例和设计报告 (包括在cow情况下的各种状态转换 (类似有限状态自动机) 的说明) 。

这个扩展练习涉及到本实验和上一个实验“虚拟内存管理”。在ucore操作系统中, 当一个用户父进程创建自己的子进程时, 父进程会把其申请的用户空间设置为只读, 子进程可共享父进程占用的用户内存空间中的页面 (这就是一个共享的资源)。当其中任何一个进程修改此用户内存空间中的某页面时, ucore会通过page fault异常获知该操作, 并完成拷贝内存页面, 使得两个进程都有各自的内存页面。这样一个进程所做的修改不会被另外一个进程可见了。请在ucore中实现这样的COW机制。

由于COW实现比较复杂, 容易引入bug, 请参考 <https://dirtycow.ninja/> 看看能否在ucore的COW实现中模拟这个错误和解决方案。需要有解释。

这是一个big challenge.

### 该Challenge未实现

### 2. 说明该用户程序是何时被预先加载到内存中的? 与我们常用操作系统的加载有何区别, 原因是什么?

解答：

### (1) 该用户程序的加载方式

在ucore的实现中，程序在do\_execve函数中被显式加载到内存。调用load\_icode函数，依次将ELF文件的文件头、程序头、程序段内容按页加载到内存空间中。在程序执行之前，所有需要的内容都已经加载到了内存中。

### (2) 常见操作系统（如Linux、Windows）的加载方式

传统操作系统广泛使用懒加载（Lazy Loading）机制，在execve中，只会加载文件头、程序头表，并分配内存区域，但此时并不会将程序段内容（代码段、数据段等）加载过来。

当程序开始执行时，以“按需加载”的方式，访问到某个虚拟地址时，若还没映射到物理地址，会通过缺页异常，从磁盘加载对应的页面内容，并更新页表。

### (3) 比较

1. ucore的一次性加载简化了内存管理，更适合嵌入式系统和教学应用；
2. 懒加载的方式减少了初始内存占用；在执行过程中，仅加载执行路径中实际需要的段，减少了内存占用和磁盘I/O。有助于提高整体性能。同时需要完善的页表和缺页异常处理机制来支持。

## LAB2分支任务实现：

---

### 基本流程思路

需要开启三个终端窗口同时进行：

- 终端 A：运行 QEMU 模拟器
- 终端 B：附加调试 QEMU 进程
- 终端 C：调试 ucore 内核

### 具体流程

1. 终端 A 运行 `make debug`。

```

rowin@Rowin:~/Documents/oslab$ cd lab2
rowin@Rowin:~/Documents/oslab/lab2$ make debug

OpenSBI v0.4 (Jul 2 2019 11:53:53)

 _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
 / \ / \ / \ / \ / \ / \ / \
 /_____\ /_____\ /_____\ /_____\ /_____\ /_____\ /_____\
 / / / / / / /
 / / / / / /
 / / / / / /
/ / / / / /
\ \ \ \ \ \
 \ \ \ \ \ \
 \ \ \ \ \ \ \
 \ \ \ \ \ \ \ \
 _____\ _____\ _____\ _____\ _____\ _____\ _____\ _____\
 \ \ \ \ \ \ \ \ \ \ \ \ \ \ \ \
 _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

Platform Name : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs : 8
Current Hart : 0
Firmware Base : 0x80000000
Firmware Size : 112 KB
Runtime SBI Version : 0.1

PMP0: 0x0000000080000000-0x000000008001ffff (A)
PMP1: 0x0000000000000000-0xffffffffffffff (A,R,W,X)
DTB Init
HartID: 0
DTB Address: 0x82200000
Physical Memory from DTB:
 Base: 0x0000000080000000
 Size: 0x0000000080000000 (128 MB)
 End: 0x0000000087ffffff
DTB init completed
(THU.CST) os is loading ...
Special kernel symbols:
 entry 0xffffffffc02000d8 (virtual)
 etext 0xffffffffc020165e (virtual)
 edata 0xffffffffc0205018 (virtual)
 end 0xffffffffc0205078 (virtual)
Kernel executable memory footprint: 20KB
memory management: best_fit_pmm_manager
physcial memory map:
 memory: 0x0000000080000000, [0x0000000080000000, 0x0000000087ffffff].
check_alloc_page() succeeded!
satp virtual address: 0xffffffffc0204000
satp physical address: 0x0000000080204000

```

2. 终端 B 通过 `pgrep -f qemu-system-riscv64` 指令获取 QEMU 进程 PID，然后使用 `sudo gdb` 命令进入宿主机 GDB。

```

rowin@Rowin:~/Documents/oslab/lab2$ pgrep -f qemu-system-riscv64
1584
rowin@Rowin:~/Documents/oslab/lab2$ sudo gdb
[sudo] password for rowin:
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
--Type <RET> for more, q to quit, c to continue without paging--

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) attach 1584
Attaching to process 1584
[New LWP 1586]
[New LWP 1585]
warning: could not find '.gnu_debugaltlink' file for /lib/x86_64-linux-gnu/libglib-2.0.so.0
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
0x00007635cc11ba30 in __GI_ppoll (
 fds=0x628a6988fab0, nfds=6,
 timeout=<optimized out>, sigmask=0x0)
 at ../sysdeps/unix/sysv/linux/ppoll.c:42
warning: 42 ../sysdeps/unix/sysv/linux/ppoll.c: No such file or directory
(gdb) handle SIGPIPE nostop noprint
Signal Stop Print Pass to program Description
SIGPIPE No No Yes Broken pipe
(gdb) c
Continuing.
^C
Thread 1 "qemu-system-ris" received signal SIGINT, Interrupt.
0x00007635cc11ba30 in __GI_ppoll (
 fds=0x628a6988fab0, nfds=6,
 timeout=<optimized out>, sigmask=0x0)
 at ../sysdeps/unix/sysv/linux/ppoll.c:42
42 in ../sysdeps/unix/sysv/linux/ppoll.c

```

3. 终端 C 运行 `make gdb`，通过在 GDB 中设置 `set remotetimeout unlimited` 对齐时序。
4. 在终端 B 中通过 `attach` 命令附加到 QEMU 进程，然后输入 `handle SIGPIPE nostop noprint` 命令，避免 QEMU 在调试过程中因信号处理而中断，接着输入 `c` 开始运行。
5. 在终端 C 中通过 `b kern_init` 设置内核入口的断点，然后输入 `c` 运行至断点处。

```

● rowin@Rowin:~/Documents/oslab/lab2$ make gdb
riscv64-unknown-elf-gdb \
 -ex 'file bin/kernel' \
 -ex 'set arch riscv:rv64' \
 -ex 'target remote localhost:1234'
GNU gdb (SiFive GDB-Metal 10.1.0-2020.12.7) 10.1
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-linux-gnu --target=riscv64-unknown-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://github.com/sifive/freedom-tools/issues>.
Find the GDB manual and other documentation resources online at:
 <http://www.gnu.org/software/gdb/documen<
Continue without paging--
tation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
Reading symbols from bin/kernel...
The target architecture is set to "riscv:rv64".
Remote debugging using localhost:1234
0x0000000000000100 in ?? ()
(gdb) set remotetimeout unlimited
(gdb) break kern_init
Breakpoint 1 at 0xffffffffc02000d8: file kern/init/init.c, line 30.
(gdb) c
Continuing.

Breakpoint 1, kern_init ()
 at kern/init/init.c:30
30 memset(edata, 0, end - edata);

```

6. 在终端 C 中通过 `x/8i $pc` 查看当前地址，并找到访存指令的位置，然后使用 `si` 指令运行到其前一句。



```

rowin@Rowin:~/Documents/oslab/lab2$ make gdb
(gdb) x/8i $pc
=> 0xffffffffc02000d8 <kern_init>:
 auipc a0,0x5
 0xffffffffc02000dc <kern_init+4>:
 addi a0,a0,-192
 0xffffffffc02000e0 <kern_init+8>:
 auipc a2,0x5
 0xffffffffc02000e4 <kern_init+12>:
 addi a2,a2,-104
 0xffffffffc02000e8 <kern_init+16>:
 addi sp,sp,-16
 0xffffffffc02000ea <kern_init+18>:
 sub a2,a2,a0
 0xffffffffc02000ec <kern_init+20>:
 li a1,0
 0xffffffffc02000ee <kern_init+22>:
 sd ra,8(sp)
(gdb) si
0xffffffffc02000dc 30 memset(edata, 0, end - edata);
(gdb)
0xffffffffc02000e0 30 memset(edata, 0, end - edata);
(gdb)
0xffffffffc02000e4 30 memset(edata, 0, end - edata);
(gdb)
0xffffffffc02000e8 28 int kern_init(void) {
(gdb)
0xffffffffc02000ea 30 memset(edata, 0, end - edata);
(gdb)
0xffffffffc02000ec 30 memset(edata, 0, end - edata);
(gdb)
0xffffffffc02000ee 28 int kern_init(void) {
(gdb) x/8i $pc
=> 0xffffffffc02000ee <kern_init+22>:
 sd ra,8(sp)
 0xffffffffc02000f0 <kern_init+24>:
 jal ra,0xffffffffc020164c <memset>
 0xffffffffc02000f4 <kern_init+28>:
 jal ra,0xffffffffc0200220 <dtb_init>
 0xffffffffc02000f8 <kern_init+32>:
 jal ra,0xffffffffc0200216 <cons_init>
 0xffffffffc02000fc <kern_init+36>:
 auipc a0,0x1
 0xffffffffc0200100 <kern_init+40>:
 addi a0,a0,1588

```

7. 在终端 B 中输入 `Ctrl-C` 中断运行，然后通过 `b get_physical_address` 设置断点，继续运行，并通过 `p/x addr` 查看当前地址。

```

rowin@Rowin:~/Documents/oslab/lab2$ sudo gdb
(gdb) b get_physical_address
Breakpoint 1 at 0x628a55f0cac3: file /home/rowin/Documents/qemu/target/riscv/cpu_helper.c, line 158.
(gdb) c
Continuing.

Thread 1 "qemu-system-ris" hit Breakpoint 1, get_physical_address (env=0x628a6984c040,
 physical=0x7ffd6f201f40,
 prot=0x7ffd6f201f40,
 addr=18446744072637907160,
 access_type=0, mmu_idx=1)
 at /home/rowin/Documents/qemu/target/riscv/cpu_helper.c:158
158 {
(gdb) p/x addr
$1 = 0xffffffffc02000d8
(gdb) c
Continuing.
[Switching to Thread 0x7635c977d6c0 (LWP 1586)]

Thread 2 "qemu-system-ris" hit Breakpoint 1, get_physical_address (env=0x628a6984c040,
 physical=0x7635c977c290,
 prot=0x7635c977c284,
 addr=18446744072637923320,
 access_type=1, mmu_idx=1)
 at /home/rowin/Documents/qemu/target/riscv/cpu_helper.c:158
158 {
(gdb) p/x addr
$2 = 0xffffffffc0203ff8
(gdb) c
Continuing.
[Switching to Thread 0x7635cc272f40 (LWP 1584)]

Thread 1 "qemu-system-ris" hit Breakpoint 1, get_physical_address (env=0x628a6984c040, physical=0x7ffd6f201f80, prot=0x7ffd6f201f80, addr=18446744072637919232, access_type=0, mmu_idx=1)
 at /home/rowin/Documents/qemu/target/riscv/cpu_helper.c:158
158 {
(gdb) c
Continuing.

Thread 1 "qemu-system-ris" hit Breakpoint 1, get_physical_address (env=0x628a6984c040, physical=0x7ffd6f201f28, prot=0x7ffd6f201f20, addr=18446744072637907160, access_type=0, mmu_idx=1)
 at /home/rowin/Documents/qemu/target/riscv/cpu_helper.c:158
158 {
(gdb) c
Continuing.
[Switching to Thread 0x7635c977d6c0 (LWP 1586)]

```

- 当终端 B 运行结束后，在终端 C 中输入 `si` 单步执行，然后输入 `i r sp` 查看地址。
- 可以看到第 7 步得到的地址为第 8 步得到的地址向后偏移了 8 个字节。

```
rowin@Rowin:~/Documents/oslab/lab2$ make gdb
(gdb) x/8i $pc
=> 0xffffffffc02000d8 <kern_init>:
 auipc a0,0x5
0xffffffffc02000dc <kern_init+4>:
 addi a0,a0,-192
0xffffffffc02000e0 <kern_init+8>:
 auipc a2,0x5
0xffffffffc02000e4 <kern_init+12>:
 addi a2,a2,-104
0xffffffffc02000e8 <kern_init+16>:
 addi sp,sp,-16
0xffffffffc02000ea <kern_init+18>:
 sub a2,a2,a0
0xffffffffc02000ec <kern_init+20>:
 li a1,0
0xffffffffc02000ee <kern_init+22>:
 sd ra,8(sp)
(gdb) si
0xffffffffc02000dc 30 memset(edata, 0, end - edata);
(gdb)
0xffffffffc02000e0 30 memset(edata, 0, end - edata);
(gdb)
0xffffffffc02000e4 30 memset(edata, 0, end - edata);
(gdb)
0xffffffffc02000e8 28 int kern_init(void) {
(gdb)
0xffffffffc02000ea 30 memset(edata, 0, end - edata);
(gdb)
0xffffffffc02000ec 30 memset(edata, 0, end - edata);
(gdb)
0xffffffffc02000ee 28 int kern_init(void) {
(gdb) x/8i $pc
=> 0xffffffffc02000ee <kern_init+22>:
 sd ra,8(sp)
0xffffffffc02000f0 <kern_init+24>:
 jal ra,0xffffffffc020164c <memset>
0xffffffffc02000f4 <kern_init+28>:
 jal ra,0xffffffffc0200220 <dtb_init>
0xffffffffc02000f8 <kern_init+32>:
 jal ra,0xffffffffc0200216 <cons_init>
0xffffffffc02000fc <kern_init+36>:
 auipc a0,0x1
0xffffffffc0200100 <kern_init+40>:
 addi a0,a0,1588
```

## 实验收获

通过这个实验可以：

1. 深入理解虚拟内存管理机制
2. 观察软件如何模拟硬件行为
3. 体验复杂系统的调试方法
4. 提高使用AI辅助学习的能力

这种“俄罗斯套娃”式的调试方式展现了现代计算机系统的层次化特性，帮助学生建立对系统底层工作机制的直观认识。



```

rowin@Rowin:~/Documents/oslab/lab5$ pgrep -f qemu-system-riscv64
9998
rowin@Rowin:~/Documents/oslab/lab5$ sudo gdb
[sudo] password for rowin:
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
--Type <RET> for more, q to quit, c to continue without paging--
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) attach 9998
Attaching to process 9998
[New LWP 10000]
[New LWP 9999]
warning: could not find '.gnu_debugaltlink' file for /lib/x86_64-linux-gnu/libglib-2.0.so.0
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
0x000078a28991ba30 in __GI_ppoll (
 fds=0x610a8c907ab0, nfds=6,
 timeout=<optimized out>, sigmask=0x0)
 at ../sysdeps/unix/sysv/linux/ppoll.c:42
warning: 42 ../sysdeps/unix/sysv/linux/ppoll.c: No such file or directory
(gdb) set remotetimeout unlimited
(gdb) handle SIGPIPE nostop noprint
Signal Stop Print Pass to program Description
SIGPIPE No No Yes Broken pipe
(gdb) c
Continuing.
^C
Thread 1 "qemu-system-ris" received signal SIGINT, Interrupt.
0x000078a28991ba30 in __GI_ppoll (
 fds=0x610a8c907ab0, nfds=6,
 timeout=<optimized out>, sigmask=0x0)
 at ../sysdeps/unix/sysv/linux/ppoll.c:42
42 in ../sysdeps/unix/sysv/linux/ppoll.c

```

3. 终端 C 运行 `make gdb`，通过在 GDB 中设置 `set remotetimeout unlimited` 对齐时序。
4. 在终端 B 中通过 `attach` 命令附加到 QEMU 进程，然后输入 `handle SIGPIPE nostop noprint` 命令，避免 QEMU 在调试过程中因信号处理而中断，接着输入 `c` 开始运行。
5. 在终端 C 中通过 `add-symbol-file obj/__user_exit.out` 加载用户程序的符号表。
6. 在终端 C 中通过 `b user/libs/syscall.c:18` 设置断点，并输入 `c` 运行。

```

○ rowin@Rowin:~/Documents/oslab/lab5$ make gdb
riscv64-unknown-elf-gdb \
 -ex 'file bin/kernel' \
 -ex 'set arch riscv:rv64' \
 -ex 'target remote localhost:1234'
GNU gdb (SiFive GDB-Metal 10.1.0-2020.12.7) 10.1
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-linux-gnu --target=riscv64-unknown-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/documentation/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
Reading symbols from bin/kernel...
The target architecture is set to "riscv:rv64".
Remote debugging using localhost:1234
0x0000000000000100 in ?? ()
(gdb) set remotetimeout unlimited
(gdb) add-symbol-file obj/__user_exit.out
add symbol table from file "obj/__user_exit.out"
(y or n) y
Reading symbols from obj/__user_exit.out...
(gdb) break user/libs/syscall.c:18
Breakpoint 1 at 0x8000f8: file user/libs/syscall.c, line 19.
(gdb) c
Continuing.

```

7. 在终端 C 中通过 `x/8i $pc` 查看当前地址，并找到 `ecall` 指令的位置，然后使用 `si` 指令运行到其前一句。

```
rowin@Rowin:~/Documents/oslab/lab5$ make gdb
```

```
Breakpoint 1, syscall (num=2)
 at user/libs/syscall.c:19
```

```
19 asm volatile (
```

```
(gdb) x/8i $pc
```

```
=> 0x8000f8 <syscall+32>:
```

```
 ld a0,8(sp)
```

```
0x8000fa <syscall+34>:
```

```
 ld a1,40(sp)
```

```
0x8000fc <syscall+36>:
```

```
 ld a2,48(sp)
```

```
0x8000fe <syscall+38>:
```

```
 ld a3,56(sp)
```

```
0x800100 <syscall+40>:
```

```
 ld a4,64(sp)
```

```
0x800102 <syscall+42>:
```

```
 ld a5,72(sp)
```

```
0x800104 <syscall+44>: ecall
```

```
0x800108 <syscall+48>:
```

```
 sd a0,28(sp)
```

```
(gdb) si
```

```
0x00000000008000fa 19 asm volatile (
```

```
(gdb)
```

```
0x00000000008000fc 19 asm volatile (
```

```
(gdb)
```

```
0x00000000008000fe 19 asm volatile (
```

```
(gdb)
```

```
0x0000000000800100 19 asm volatile (
```

```
(gdb)
```

```
0x0000000000800102 19 asm volatile (
```

```
(gdb)
```

```
0x0000000000800104 19 asm volatile (
```

```
(gdb) x/8i $pc
```

```
=> 0x800104 <syscall+44>: ecall
```

```
0x800108 <syscall+48>:
```

```
 sd a0,28(sp)
```

```
0x80010c <syscall+52>:
```

```
 lw a0,28(sp)
```

```
0x80010e <syscall+54>:
```

```
 addi sp,sp,144
```

```
0x800110 <syscall+56>: ret
```

```
0x800112 <sys_exit>: mv a1,a0
```

```
0x800114 <sys_exit+2>:
```

```
 li a0,1
```

```
0x800116 <sys_exit+4>:
```

8. 在终端 B 中输入 `Ctrl-C` 中断运行，然后通过 `b riscv_cpu_do_interrupt` 设置断点，输入 `c` 继续运行。

```

rowin@Rowin:~/Documents/oslab/lab5$ sudo gdb
timeout=<optimized out>, sigmask=0x0)
at ../sysdeps/unix/sysv/linux/ppoll.c:42
warning: 42 ../sysdeps/unix/sysv/linux/ppoll.c: No such file or directory
(gdb) set remotetimeout unlimited
(gdb) handle SIGPIPE nostop noprint
Signal Stop Print Pass to program Description
SIGPIPE No No Yes Broken pipe
(gdb) c
Continuing.
^C
Thread 1 "qemu-system-ris" received signal SIGINT, Interrupt.
0x000078a28991ba30 in __GI_ppoll (
 fds=0x610a8c907ab0, nfd=6,
 timeout=<optimized out>, sigmask=0x0)
 at ../sysdeps/unix/sysv/linux/ppoll.c:42
42 in ../sysdeps/unix/sysv/linux/ppoll.c
(gdb) break riscv_cpu_do_interrupt
Breakpoint 1 at 0x610a662b5d37: file /home/rowin/Documents/qemu/target/riscv/cpu_helper.c, line 507.
(gdb) c
Continuing.
[Switching to Thread 0x78a288ee16c0 (LWP 10000)]

Thread 2 "qemu-system-ris" hit Breakpoint 1, riscv_cpu_do_interrupt (cs=0x610a8c8bb630)
 at /home/rowin/Documents/qemu/target/riscv/cpu_helper.c:507
507 RISCV_CPU *cpu = RISCV_CPU(cs);
(gdb) c
Continuing.

Thread 2 "qemu-system-ris" hit Breakpoint 1, riscv_cpu_do_interrupt (cs=0x610a8c8bb630)
 at /home/rowin/Documents/qemu/target/riscv/cpu_helper.c:507
507 RISCV_CPU *cpu = RISCV_CPU(cs);
(gdb) c
Continuing.
^C
Thread 1 "qemu-system-ris" received signal SIGINT, Interrupt.
[Switching to Thread 0x78a289764f40 (LWP 9998)]
0x000078a28991ba30 in __GI_ppoll (
 fds=0x610a8c907ab0, nfd=6,
 timeout=<optimized out>, sigmask=0x0)
 at ../sysdeps/unix/sysv/linux/ppoll.c:42
warning: 42 ../sysdeps/unix/sysv/linux/ppoll.c: No such file or directory
(gdb) c
Continuing.
(gdb)

```

9. 在终端 C 中输入 `si` 单步执行，处理 `ecall` 指令。
10. 在终端 B 中输入 `c` 继续执行。
11. 在终端 C 中通过 `b kern/trap/trapentry.S:133` 设置断点，并输入 `c` 运行。
12. 在终端 C 中通过 `x/8i $pc` 查看当前地址，发现程序成功地停在了 `sret` 指令的前一句。

```

rowin@Rowin:~/Documents/oslab/lab5$ make gdb
(gdb) i r pc
pc 0x800104 0x800104 <syscall+44>
(gdb) si
^C^CThe target is not responding to interrupt requests.
Stop debugging it? (y or n) n

Program received signal SIGINT, Interrupt.
__alltraps () at kern/trap/trapentry.S:123
123 SAVE_ALL
(gdb) b trapentry.S:133
Breakpoint 2 at 0xffffffffc0200eca: file kern/trap/trapentry.S, line 133.
(gdb) x/8i $pc
=> 0xffffffffc0200e04 <__alltraps>:
 csrrw sp,sscratch,sp
0xffffffffc0200e08 <__alltraps+4>:
 bnez sp,0xffffffffc0200e10 <__alltraps+12>
0xffffffffc0200e0c <__alltraps+8>:
 csrr sp,sscratch
0xffffffffc0200e10 <__alltraps+12>:
 addi sp,sp,-288
0xffffffffc0200e12 <__alltraps+14>:
 sd zero,0(sp)
0xffffffffc0200e14 <__alltraps+16>:
 sd ra,8(sp)
0xffffffffc0200e16 <__alltraps+18>:
 sd gp,24(sp)
0xffffffffc0200e18 <__alltraps+20>:
 sd tp,32(sp)
(gdb) c
Continuing.

Breakpoint 2, __trapret ()
 at kern/trap/trapentry.S:133
133 sret
(gdb) x/8i $pc
=> 0xffffffffc0200eca <__trapret+86>:
 sret
0xffffffffc0200ece <forkrets>:
 mv sp,a0
0xffffffffc0200ed0 <forkrets+2>:
 j 0xffffffffc0200e74 <__trapret>
0xffffffffc0200ed2 <kernel_execve_ret>: addi a1,a1,-288
0xffffffffc0200ed6 <kernel_execve_ret+4>:ld s1,280(a0)
0xffffffffc0200eda <kernel_execve_ret+8>:sd s1,280(a1)

```

13. 在终端 B 中设置断点，输入 `c` 继续运行。

14. 在终端 C 中输入 `si` 单步执行，处理 `sret` 指令。



```

rowin@Rowin:~/Documents/oslab/lab5$ make gdb
__alltraps () at kern/trap/trapentry.S:123
123 SAVE_ALL
(gdb) b trapentry.S:133
Breakpoint 2 at 0xffffffffc0200eca: file kern/trap/trapentry.S, line 133.
(gdb) x/8i $pc
=> 0xffffffffc0200e04 <__alltraps>:
 csrrw sp,sscratch,sp
0xffffffffc0200e08 <__alltraps+4>:
 bnez sp,0xffffffffc0200e10 <__alltraps+12>
0xffffffffc0200e0c <__alltraps+8>:
 csrr sp,sscratch
0xffffffffc0200e10 <__alltraps+12>:
 addi sp,sp,-288
0xffffffffc0200e12 <__alltraps+14>:
 sd zero,0(sp)
0xffffffffc0200e14 <__alltraps+16>:
 sd ra,8(sp)
0xffffffffc0200e16 <__alltraps+18>:
 sd gp,24(sp)
0xffffffffc0200e18 <__alltraps+20>:
 sd tp,32(sp)
(gdb) c
Continuing.

Breakpoint 2, __trapret ()
 at kern/trap/trapentry.S:133
133 sret
(gdb) x/8i $pc
=> 0xffffffffc0200eca <__trapret+86>:
 sret
0xffffffffc0200ece <forkrets>:
 mv sp,a0
0xffffffffc0200ed0 <forkrets+2>:
 j 0xffffffffc0200e74 <__trapret>
0xffffffffc0200ed2 <kernel_execve_ret>: addi a1,a1,-288
0xffffffffc0200ed6 <kernel_execve_ret+4>:ld s1,280(a0)
0xffffffffc0200eda <kernel_execve_ret+8>:sd s1,280(a1)
0xffffffffc0200ede <kernel_execve_ret+12>:ld s1,272(a0)
0xffffffffc0200ee2 <kernel_execve_ret+16>:sd s1,272(a1)
(gdb) si
0x0000000000800108 in syscall (num=3)
 at user/libs/syscall.c:19
19 asm volatile (
(gdb)

```

## 实验收获与理解

通过Lab5分支任务，可以获得以下深入理解：

1. **系统调用完整流程**：从用户态发起系统调用，通过 `eca11` 进入内核态处理，再通过 `sret` 返回用户态的全过程
2. **特权级切换机制**：理解RISC-V架构中U-mode与S-mode之间的切换及状态保存/恢复机制
3. **QEMU模拟精度**：观察到QEMU如何精确模拟硬件行为，包括CSR寄存器操作和异常处理流程
4. **软件与硬件的关系**：体验软件如何模拟硬件功能，加深对计算机系统层次化结构的理解

## 5. **调试技能提升：**掌握复杂系统调试方法，特别是双重GDB调试技术的应用

这种"俄罗斯套娃"式的调试方法展示了现代计算机系统的复杂性和层次化特征，为深入理解操作系统和计算机体系结构提供了宝贵的实践经验。